

Технически университет Варна



Курсов проект

дисциплина: Технологии на софтуерното производство

специалност: Софтуерни и интернет технологии

тема: Система за обяви за работа

Разработващ екип:

- | | |
|---------------------|----------|
| 1. Яна Ампова | 22621751 |
| 2. Теодора Стойчева | 22621693 |

2025

1. Задание

Разработваната система е уеб-базирана платформа, която да обслужва процеса по търсене и кандидатстване за работа от страна на индивидуални потребители (кандидати). Основната цел на платформата е да осигури лесен, структуриран и интуитивен достъп до публикувани обяви за работа, като същевременно позволи на потребителите да създадат и поддържат собствен профил, с който да управляват своите кандидатури. Платформата също така предоставя на потребители-фирми възможност за публикуване и управление на обяви за работа, проследяване на кандидатури, достъп до статистики и конфигуриране на фирмения акаунт.

1.1. Потребителска регистрация и профил

- Потребителите трябва да могат да създадат акаунт чрез регистрационна форма.
- След регистрация трябва да е налична възможност за редактиране на профила, включително:
- Добавяне/промяна на име, контактна информация, кратко описание.
- Прикачване на автобиография (резюме) във вид на документ.
- Добавяне на профилна снимка.
- Въвеждане и редактиране на списък с умения и компетенции.
- Фирмите трябва да могат да създават фирмен акаунт с възможност за въвеждане на информация за компанията, лого, контактни данни и описание на дейността.

1.2. Обяви за работа

- Платформата трябва да предоставя списък с налични обяви за работа.
- Всяка обява трябва да съдържа информация за позицията, компанията, местоположението, изискванията, предлаганите условия и бенефити.
- Потребителите трябва да могат да разглеждат детайлно всяка обява.
- Фирмите трябва да имат възможност да създават, редактират и изтриват свои обяви, както и да задават срок за кандидатстване и специфични критерии.

1.3. Филтриране и търсене

- Да се реализира функционалност за търсене и филтриране на обяви по различни критерии, включително:
- Град / местоположение
- Сфера / индустрия
- Ниво на позицията (начално, средно, мениджърско и др.)
- Изисквания към кандидата (опит, умения и др.)
- Предлагани бонуси към работната позиция
- Фирмите трябва да имат възможност да търсят и филтрират между подадените кандидатури по критерии като умения, статус на кандидатурата и предишен опит.

1.4. Действия с обяви

- Потребителите трябва да могат да добавят обяви към списък "Любими" за бъдещо разглеждане.
- Възможност за премахване на обяви от този списък.

- Всеки потребител трябва да има възможност да кандидатства директно по обява.
- При кандидатстване системата да използва прикачената автобиография от профила на потребителя.
- Да се регистрира и запазва информацията за подадената кандидатура.
- В профила на потребителя трябва да бъде наличен раздел за преглед на всички подадени кандидатури.
- Всяка кандидатура трябва да съдържа информация за съответната обява и статус (например: „подадена“, „разгледана“, „отказана“, „одобрена“ и др.).
- Фирмите трябва да могат да преглеждат получените кандидатури по техни обяви, да виждат подробна информация за кандидатите и да актуализират статуса на всяка кандидатура.

Платформата трябва да бъде достъпна чрез уеб браузър, с ясен и интуитивен потребителски интерфейс, насочен към крайния потребител – търсещ работа.

2. Анотация

Проектът представлява разработка на уеб-базирана система за управление на процеса по търсене и кандидатстване за работа от страна на индивидуални потребители. Основната идея зад платформата е да улесни кандидатите в ориентирането сред множество обяви чрез прецизно филтриране по различни критерии и възможност за структурирано управление на техния профил и кандидатури.

Проектът обхваща няколко основни модула, които са реализирани като отделни, но взаимосвързани компоненти. На първо място е изградена система за регистрация и управление на потребителски профил, която позволява добавяне на лична и професионална информация, както и прикачване на документи като автобиография и снимка. Следващият ключов модул е свързан с визуализиране и търсене на работни обяви – реализирани са функционалности за динамично филтриране по различни параметри като местоположение, сфера на дейност, изисквания към кандидата и други.

Кандидатстването по обяви се осъществява директно от системата, като потребителят може да избира и проследява статусите на своите кандидатури в специален раздел. Добавена е и възможност за съхраняване на избрани обяви в списък с предпочитания („любими“), което улеснява бъдещ достъп и сравнение между предложенията.

Успоредно с функционалностите за индивидуални кандидати, платформата включва и цялостен модул за потребители-фирми, който позволява публикуване, редактиране и изтриване на обяви, управление на постъпилите кандидатури, промяна на техния статус и преглед на подробна информация за всеки кандидат. Предвиден е и административен панел със статистики и настройки на фирмения акаунт, което подпомага работодателите в организиране на подбора и анализ на ефективността на публикуваните позиции.

В резултат на проекта е реализирана пълноценна и работеща уеб платформа, която обединява процеса по създаване на професионален профил, филтриране на работни позиции и директно кандидатстване, с възможност за бъдещо разширяване и адаптиране към нуждите на реална бизнес среда. Проектът демонстрира приложимостта на концепцията за дигитализация на търсенето на работа чрез интуитивен и фокусиран потребителски интерфейс.

3. Увод

В последното десетилетие се наблюдава значително нарастване на дигитализацията в различни сектори на обществения и икономическия живот. Един от най-засегнатите процеси в тази трансформация е процесът по търсене и предлагане на работа. Все повече компании използват онлайн платформи за публикуване на обяви, а кандидатите разчитат изцяло на уеб-базирани решения при намирането на подходящи позиции. На фона на тази тенденция нуждата от ефективни, бързи и интуитивни инструменти за управление на кандидатури става все по-съществена.

Актуалността на темата е пряко свързана с динамиката на съвременния пазар на труда, който изисква от търсещите работа гъвкавост, бързина и добра информираност. В този контекст платформи, които предлагат ясна структура, възможност за лесно търсене по ключови критерии и управление на личен профил, се превръщат в основен инструмент в процеса по кандидатстване. Въпреки наличието на редица подобни решения, съществуват редица предизвикателства, като усложнен интерфейс, платени услуги или ограничена ориентация към крайния потребител.

Предложеният проект се стреми да отговори именно на тези нужди, като предложи решение, фокусирано изцяло върху потребителското преживяване на кандидата. Разработката цели да съчетае удобство, бързина и практичност чрез интуитивна система за регистрация, търсене и кандидатстване по работни позиции. Така проектът не само е технически предизвикателен, но и социално значим, тъй като отговаря на реална и нарастваща необходимост на пазара.

4. Обзор на съществуващи решения. Изводи. Цели и задачи

4.1. Анализ на съществуващи решения

На българския пазар съществуват утвърдени платформи за публикуване и търсене на работа, сред които водещи примери са Jobs.bg и Dev.bg.

Jobs.bg е най-популярната платформа за обяви за работа в България, предлагаща широк обхват от позиции във всички икономически сектори. Платформата разполага с мощна система за филтриране, автоматични известия, възможност за създаване на автобиография онлайн и управление на кандидатури. Въпреки това, интерфейсът на сайта често е критикуван за претрупаност и затруднена ориентация при първоначално използване. Част от функционалностите изискват създаване на профил или преминаване през няколко стъпки, което може да бъде демотивиращо за някои потребители.

Dev.bg от своя страна е специализирана платформа, фокусирана върху обяви за работа в IT сектора. Тя предлага съвременен и изчистен интерфейс, по-ограничен брой, но внимателно курирани обяви, както и допълнителна стойност чрез статии, новини и общностни събития. Въпреки това, платформата не предоставя достатъчно възможности за персонализиране на потребителския профил или проследяване на кандидатурите, а обхватът ѝ е ограничен главно до технологичната сфера.

4.2. Изводи

На база на направения анализ могат да се открият следните предимства и недостатъци:

Предимства на съществуващите решения:

- Широк обхват от обяви (Jobs.bg)
- Специализация и съдържателен фокус (Dev.bg)
- Добре развита система за филтриране (и при двете платформи)
- Възможност за онлайн кандидатстване и запазване на обяви

Недостатъци:

- Сложен и претрупан интерфейс (Jobs.bg)
- Липса на модерен UX подход и персонализация на преживяването
- Липса на ясна проследимост на статусите по кандидатури
- Ограничена ориентация към нуждите на индивидуалните кандидати (липса на профилен фокус)
- Недостатъчна функционалност при управление на кандидатствания и автобиографии

4.3. Цели

Целта на проекта Jobbu е създаването на интуитивна и фокусирана уеб платформа, ориентирана към индивидуалния кандидат за работа, която комбинира лесна

навигация, персонализирано съдържание и ефективно управление на процеса по търсене и кандидатстване по обяви.

4.4. Задачи

За постигане на тази цел трябва да бъдат реализирани следните основни задачи:

1. Разработване на модул за регистрация и поддръжка на потребителски профил с възможност за редакция на лична и професионална информация.
2. Имплементация на функционалност за прикачване и преглед на автобиография и профилна снимка.
3. Създаване на модул за визуализация и търсене на работни обяви с възможност за филтриране по различни критерии – местоположение, сфера, ниво и други.
4. Реализиране на механизъм за запазване на обяви в списък „Любими“.
5. Осигуряване на възможност за кандидатстване по обяви директно през платформата.
6. Въвеждане на панел за преглед и проследяване на кандидатурите с ясно отбелязване на техния статус.
7. Изграждане на структуриран и интуитивен потребителски интерфейс, осигуряващ бърз достъп до всички основни функционалности.
8. Разработване на модул за фирмени потребители с възможност за създаване, редактиране и изтриване на обяви за работа.
9. Имплементация на система за управление на кандидатури от страна на фирмите, включваща преглед на кандидатите и промяна на статуса на всяка кандидатура.
10. Създаване на интерфейс за преглед на фирмени статистики и настройки на фирмения акаунт.

5. Проектиране и описание на решението

5.1. Функционални изисквания към програмната система

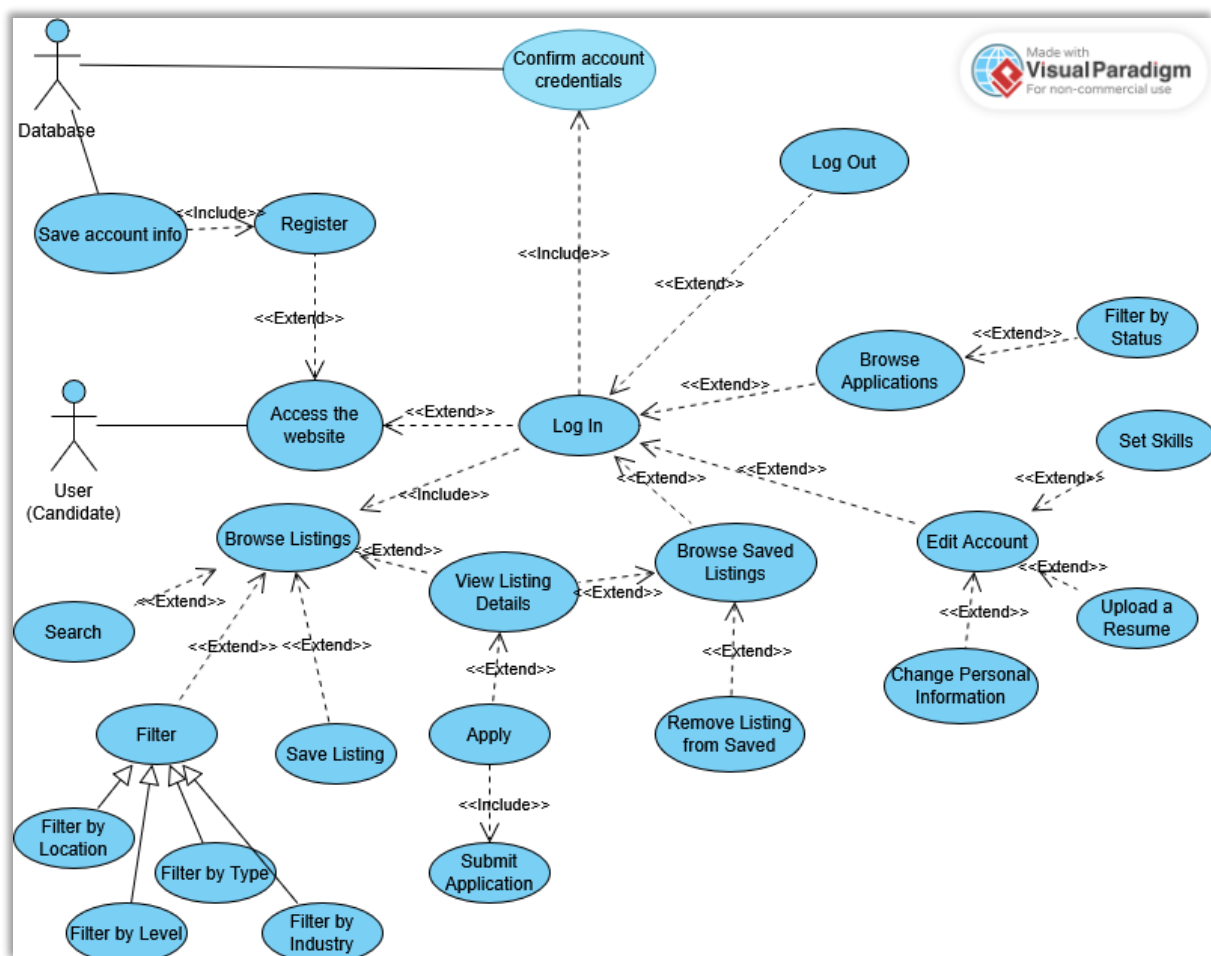
- Създаване на потребителски профил с възможност за регистрация и вход в системата.
- Редактиране на профил
- Преглед на списък с активни обяви за работа.
- Филтриране на обявите по критерии: град, сфера, изисквания, ниво на позиция, бенефити и др.
- Добавяне на обяви към списък с любими.
- Кандидатстване по обява с прикачено CV от профила.
- Преглед на подадените кандидатури с актуален статус (напр. подадена, прегледана, отхвърлена).
- Сигурно излизане от профила и защита на личните данни.
- Регистрация и достъп до фирмен профил с възможност за управление на фирмена информация и настройки.
- Създаване, редактиране и изтриване на обяви за работа от страна на фирмени потребители.
- Преглед на получени кандидатури по обяви с възможност за филтриране и промяна на техния статус.
- Достъп до статистики, свързани с активни обяви, кандидатури и одобрени кандидати.

5.2. Нефункционални изисквания към програмната система

- Достъпност: Платформата трябва да е достъпна през съвременни уеб браузъри (Chrome, Firefox, Edge).
- Сигурност: Да се гарантира защита на профилите чрез механизми за автентикация и ограничен достъп до чувствителна информация.
- Скалируемост: Системата трябва да поддържа добавяне на нови функционалности и лесна поддръжка.
- Производителност: Заявките към базата данни да се изпълняват с минимално време на реакция.
- Надеждност: Данните за потребители, кандидатури и обяви трябва да се съхраняват в защитена среда и да не се губят при срива на системата.

5.3. Логически модел

Логическият модел на платформата е представен чрез UML диаграма, която илюстрира основните действия, които кандидатът – единственият тип потребител в системата – може да извършва. Целта на модела е да визуализира функционалния обхват на системата от гледна точка на крайния потребител и да подчертае логическите връзки между отделните функционалности.



Потребителят стартира взаимодействието със системата чрез регистрация, при която въвежда необходимите лични данни. След попълване на формата за регистрация, информацията се съхранява в базата данни. Следващата стъпка е вход в системата (Log In), при който се извършва потвърждение на въведените идентификационни данни чрез механизъм за автентикация. При успешно логване потребителят получава достъп до пълния набор от функционалности на платформата.

След влизане в системата, потребителят може да разглежда публикуваните обяви за работа. Тази функционалност включва както възможност за свободно преглеждане на всички налични позиции, така и възможност за търсене и филтриране според конкретни критерии. Системата позволява филтриране по местоположение, тип на позицията, ниво и индустрия, което осигурява персонализирано потребителско преживяване и бърз достъп до най-релевантните предложения.

Всяка обява може да бъде разгледана в детайли, а ако потребителят я намери за интересна, може да я запази в списък с любими. Платформата предоставя отделна секция за преглед на тези запазени обяви, с възможност за премахване на вече нежелани позиции. В допълнение, кандидатът може директно да подаде заявление за работа по избрана обява. Процесът на кандидатстване включва подаване на заявка, която се регистрира и съхранява в системата.

Потребителят има достъп до раздел с всички свои кандидатури, където може да проследи статуса на всяка от тях – например дали е подадена, разгледана или

отхвърлена. За по-добра ориентация системата поддържа и филтриране по статус на кандидатурата.

Отделно от работата с обяви, платформата предлага възможност за редактиране на профила. Потребителят може да качи автобиография във вид на документ, да добави информация за своите умения и компетенции, както и да променя лични данни като име, контактна информация и описание. Всички тези действия се извършват през централния модул за управление на акаунта, който е достъпен след вход в системата.

Цялостната логика е изградена чрез ясно дефинирани зависимости между прецедентите. Ключовите действия като кандидатстване, запазване на обяви или редакция на профил са разширения на основните потоци, като някои от тях включват допълнителни, задължителни подфункции – например потвърждение на данни при вход или подаване на заявление при кандидатстване. Това допринася за ясно структурирана архитектура, която отразява реалното поведение на потребителя и логиката на системата по прозрачен и проследим начин.

5.4. Архитектура

Платформата Jobby е реализирана по модел на триизмерна архитектура (three-tier architecture), основана на Model–View–Controller (MVC) парадигмата. Архитектурата включва следните слоеве:

- Контролери (Controllers): обработват HTTP заявки, валидират входни данни и пренасочват логиката към съответния сървисен слой.
- Сървисен слой (Services): съдържа бизнес логиката на приложението и управлява комуникацията между контролерите и слоя за достъп до данни.
- Модели и база данни (Models/Repositories): осъществяват достъп до базата данни чрез ORM технологията Entity Framework.

За управление на потребителската автентикация се използва стандартен ASP.NET Identity, конфигуриран в отделен DbContext клас. Това осигурява изолация между модулите за сигурност и останалата част от данните. Всички данни се съхраняват в SQL Server база, хоствана в Azure Portal, което позволява онлайн достъп и лесно управление на инфраструктурата.

5.5. Избор на език и средства за програмиране

Разработката на платформата е реализирана с помощта на съвременни и широко използвани технологии, които отговарят както на функционалните изисквания на системата, така и на нефункционалните – свързани със сигурност, производителност, мащабируемост и поддръжка

Език за програмиране: C#

C# е обектно-ориентиран, строго типизиран език, който предоставя висока степен на надеждност, четимост и стабилност на кода. Той е основен език за разработка в .NET екосистемата и предлага богата библиотека от класове, подходящи за изграждане на уеб базирани приложения с комплексна логика. Изборът на C# позволява прецизно

управление на бизнес логиката, интеграция със сигурностни механизми и лесна работа с бази от данни чрез LINQ и Entity Framework.

Уеб фреймуърк: ASP.NET Core MVC

ASP.NET Core MVC е модерен и кросплатформен фреймуърк за разработка на уеб приложения, който следва архитектурния модел MVC (Model–View–Controller). Той осигурява ясно разделение между логика, изгледи и модели, което улеснява разширяемостта и поддръжката на проекта. Фреймуъркът е подходящ за създаване на уеб приложения със средна до висока сложност, каквото представлява и платформата **Jobby**. Неговата вградена поддръжка за dependency injection, middleware и сигурност прави архитектурата на приложението устойчива и гъвкава.

Шаблони и изгледи: Razor Pages

Razor Pages осигуряват лесен за управление и структуриран начин за изграждане на потребителски интерфейс. Те позволяват логиката за обработка на входа да бъде разположена близо до изгледа, което улеснява разработката на страници с фокус върху конкретна функционалност (напр. регистрация, кандидатстване, редакция на профил). Razor комбинира HTML със C# в един и същи файл, което допринася за по-добра интеграция между данни и изгледи, без да нарушава модулността на системата.

Frontend технологии: HTML5, CSS3, JavaScript

Използването на стандартните уеб технологии за клиентската част – HTML5 (структура), CSS3 (визуално оформление) и JavaScript (интерактивност) – гарантира съвместимост с всички съвременни браузъри, висока степен на персонализация на интерфейса и добра потребителска достъпност. Тези технологии позволяват реализирането на динамичен и отзивчив (responsive) интерфейс, който е от критично значение при потребителски ориентирани системи като Jobby.

Среда за разработка: Visual Studio

Visual Studio предлага пълна интеграционна среда за .NET приложения, с инструменти за разработка, отстраняване на грешки, контрол на версиите, управление на пакети и директна интеграция с Azure. Тя осигурява бърза настройка на проекти, поддръжка на Razor, Intellisense за C#, както и шаблони за ASP.NET MVC, което съкращава времето за разработка и минимизира риска от грешки.

Сървърна платформа: Azure Portal

Хостването на приложението в Azure Portal осигурява висока достъпност, автоматично мащабиране, управление на ресурси и вградени възможности за архивиране и мониторинг. Това прави проекта устойчив в реална среда и лесен за надграждане в бъдеще.

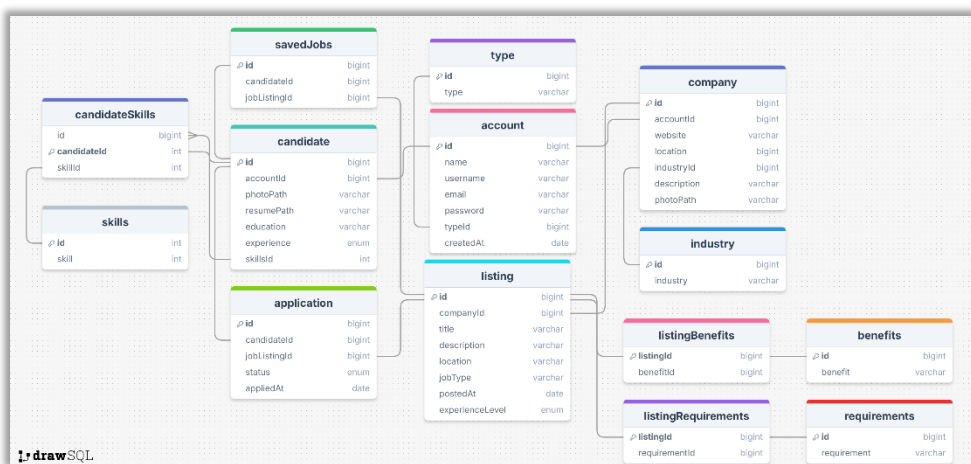
Изборът на SQL Server осигурява надеждно съхранение на данни, висока производителност и стабилност при работа с голям обем от структурирана информация. Базата е интегрирана с Azure и се управлява чрез Entity Framework, което позволява бърза ORM свързаност и лесна миграция на данни.

ORM: Entity Framework Core

Entity Framework Core улеснява взаимодействието с базата от данни чрез обектно-ориентиран подход. То премахва нуждата от директно писане на SQL заявки и поддържа миграции, LINQ заявки, валидиране на модели и автоматично картографиране на обектите. Използването му повишава продуктивността и намалява възможността за грешки при работа с данни.

5.6. Концептуален модел на база данни

Базата данни на системата представлява реляционен модел, който обхваща ключовите функционалности на платформата – потребителски профили, фирмени акаунти, обяви за работа, кандидатури, умения, бенефити и изисквания. По-долу следва подробен преглед на таблиците, техните релации и предназначение.



Към акаунтите от тип кандидат е свързана таблицата `candidate`, която съдържа допълнителна информация като път до профилна снимка (`photoPath`), автобиография (`resumePath`), образование, трудов опит (enum `experience`), както и референция към списък с умения чрез `skillsId`. Уменията се съхраняват в таблицата `skills`, а релацията между кандидат и конкретните му умения се реализира чрез междинната таблица `candidateSkills`, която съдържа `candidateId` и `skillId`.

Всеки кандидат може да кандидатства по обяви чрез таблицата `application`. Тя свързва кандидата (`candidateId`) с конкретна обява (`jobListingId`) и съдържа полета като статус на кандидатурата (enum) и дата на подаване. За улесняване на бъдещи действия, кандидатите могат да запазват обяви в списък с предпочитания, реализиран чрез таблицата `savedJobs`, която също съдържа `candidateId` и `jobListingId`.

От страна на фирмите, таблицата `company` съдържа фирмената информация, свързана с конкретен `accountId`. В нея се съхраняват уебсайт, местоположение (външен ключ към

таблицата `location`), индустрия (чрез `industryId`, външен ключ към таблицата `industry`), описание и фирмено лого. Това позволява пълноценно представяне на фирмата в платформата.

Обявите за работа са дефинирани в таблицата `listing`, която е свързана с конкретна фирма чрез `companyId`. Всяка обява съдържа заглавие, описание, местоположение (вече като външен ключ към таблицата `location`), тип на работа (`jobTypeId`, референция към таблицата `jobType`), дата на публикуване и ниво на опит (`experienceLevel`, `enum`). За по-голяма гъвкавост и разширяемост, бенефитите и изискванията към всяка обява се моделират чрез свързващи таблици: `listingBenefits` и `listingRequirements`, които съответно реферират таблиците `benefits` и `requirements`.

5.7. Реализация на програмната система

5.7.1. Структура на данните

Данните в системата се съхраняват в релационна база данни (SQL Server), организирана в таблици със строга типизация. Всяка таблица притежава първичен ключ (`id`), дефиниран като `INT` или `GUID`, в зависимост от нуждата от уникалност и разширяемост. Връзките между таблиците се осъществяват чрез външни ключове (`foreign keys`), осигуряващи референтна цялост.

Ключовите полета включват:

- В таблицата `account`: `id` (PK, `INT`), `email` (`nvarchar(255)`), `password` (`nvarchar(255)`, хеширана стойност), `typeId` (FK към `type`)
- В таблицата `candidate`: `resumePath`, `photoPath` (`nvarchar(MAX)`), `experience` (`enum` / `int`)
- В таблицата `listing`: `title` (`nvarchar(100)`), `description` (`nvarchar(MAX)`), `locationId`, `jobTypeId`, `companyId`
- В таблицата `application`: `status` (`enum`), `appliedAt` (`datetime`), `candidateId`, `listingId`

Типовете на данните са подбрани така, че да осигуряват баланс между производителност и гъвкавост (напр. `nvarchar` за текстови полета с Unicode поддръжка, `datetime` за дати, `bit` за булеви стойности).

5.7.2. Описание на програмните модули

Програмната система следва трислойна архитектура по модела MVC (Model–View–Controller), като отделните модули имат следната функционалност:

Контролери: Получават HTTP заявки, валидират данните, извикват съответните сървисни методи и връщат резултат (във вид на изглед или JSON отговор).

Сървисен слой (Services): Обработва бизнес логиката – например проверка за съществуваща кандидатура, създаване на нова обява, промяна на статус на кандидат. Всеки сървис служи като посредник между контролера и слоя с данни.

Модели и репозитории: Представяват бизнес обектите и логиката за достъп до базата чрез Entity Framework. Репозиториите реализират CRUD операции с LINQ заявки.

Показан е алгоритъм за кандидатстване по обява в псевдокод:

При POST заявка към /Apply:

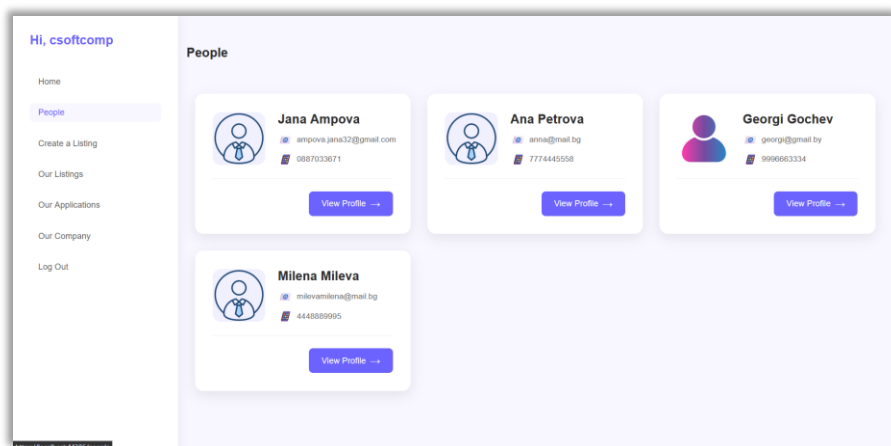
- Проверка за автентикация на потребителя.
- Валидация: дали обявата съществува и дали вече има кандидатура.
- Създаване на нова Application:
 - задаване на статус "Подадена"
 - асоцииране към кандидата и обявата
- Запис в базата
- Пренасочване към списък с кандидатури

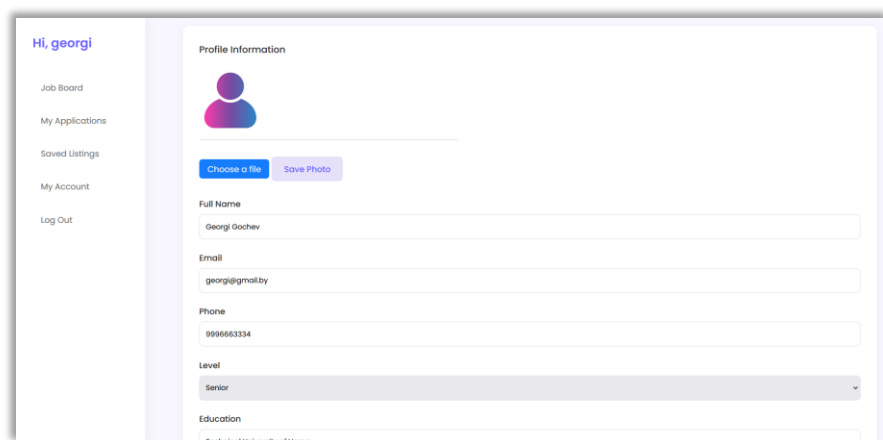
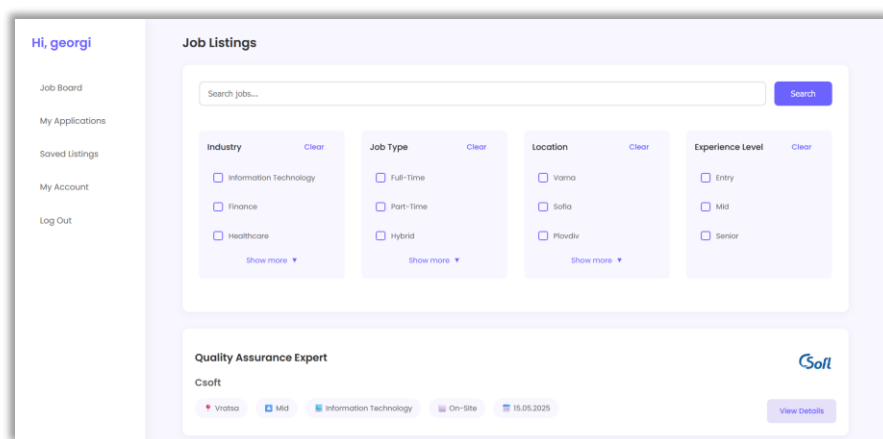
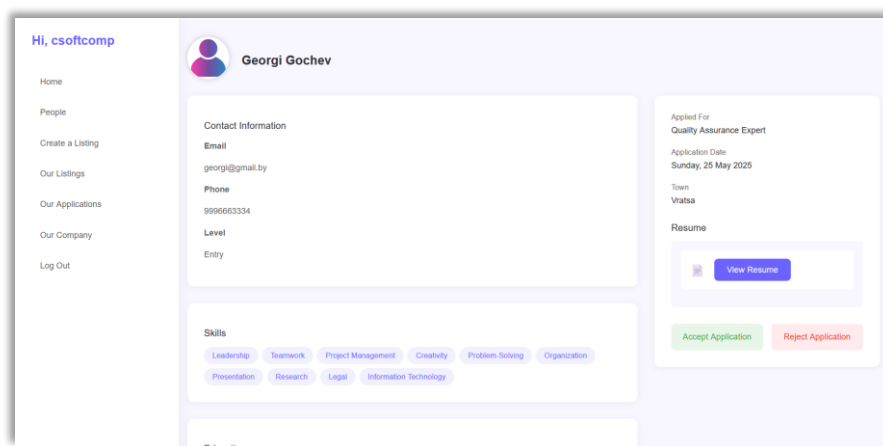
5.7.3. Организация на потребителския интерфейс

Потребителският интерфейс е изграден с Razor Pages и стандартни уеб технологии (HTML5, CSS3, JavaScript), организирани по начин, който следва ролевия модел – различен интерфейс за кандидати и фирми.

Основни елементи са:

- Навигационно меню, достъпно след вход, което се адаптира според роля: кандидатите виждат „Обяви“, „Любими“, „Кандидатури“, а фирмите – „Моите обяви“, „Кандидати“, „Статистики“.
- Формуляри с валидация: регистрация, вход, редакция на профил, публикуване на обява, кандидатстване.
- Изгледи със списъци: обяви с възможност за филтриране, таблици с кандидатури, табла със статистика.
- Адаптивен дизайн, подходящ за работа от мобилни устройства и десктоп.





5.8. Формат на входните документи

Всички входни данни в системата се въвеждат чрез уеб форми, разработени с Razor Pages и HTML5 компоненти. Формалният контрол се осъществява чрез валидация на типове данни (например имейл, дата, текст, файл) както на клиентско (JavaScript), така и на сървърно ниво (.NET моделен валидатор). Не се допускат празни задължителни полета, неправилни файлови формати (напр. CV само във формат .pdf или .docx), или въвеждане на символи, които могат да застрашат сигурността (SQL injection, XSS). Логическият контрол включва проверка на смисловата пълнота и коректност – напр. при кандидатстване се изисква предварително прикачено CV; при публикуване на обява – задължителни са полета за длъжност, локация, индустрия и описание.

Системата извежда съобщения за грешка при некоректни или непълни входни данни, и позволява корекция преди изпращане.

5.9. Формат на извежданите справки

Справките в системата са предимно визуални и достъпни през потребителския интерфейс, реализиран с Razor изгледи. За кандидатите се извеждат справки с техните кандидатури, съдържащи информация за длъжност, фирма, дата на кандидатстване и текущ статус. За фирмите се предоставят динамични списъци на подадени кандидатури по всяка обява, с възможност за филтриране и сортиране по статус, дата, или умения. Допълнително са налични обобщени справки в табличен вид: брой публикувани обяви, общ брой кандидатури, одобрени/отхвърлени кандидати и др. Всички справки се визуализират директно в браузъра, а при нужда могат да се експортират чрез вградена функционалност (напр. в .csv или .pdf – при бъдещо разширение на системата).

5.10. . Инструкции за работа

5.10.1. Ръководство за потребителя

- Потребителите получават достъп до системата чрез уеб браузър, като началната страница предоставя възможност за вход или регистрация.
- След вход в системата кандидатите могат да редактират своя профил, да прикачват документи, да търсят обяви чрез филтри и да кандидатстват директно по избрани позиции.
- Фирмените потребители имат достъп до административен панел, чрез който публикуват и управляват обяви, обработват кандидатури и следят статистика.
- Навигацията между отделните секции е осигурена чрез ясно структурирано меню и адаптивен дизайн.
- Всяка основна функционалност (регистрация, публикуване, кандидатстване) е реализирана чрез отделен изглед, базиран на Razor Pages, с вградена валидация на входа.

5.10.2. Инструкции и изисквания при инсталиране на системата

- Системата се разработва с ASP.NET Core MVC и изисква .NET SDK (версия 6.0 или по-нова) и SQL Server.
- Инсталацията включва клониране на проекта, настройка на appsettings.json с параметри за връзка към базата и изпълнение на Entity Framework миграции.
- За хостване в Azure: необходимо е създаване на App Service, SQL Database и деплой чрез GitHub или Visual Studio.
- Всички зависимости се управляват чрез NuGet, а миграциите се изпълняват чрез CLI или Visual Studio Package Manager Console.

5.10.3 Инструкции за поддържане на системата

- Поддръжката включва редовно архивиране на базата данни, мониторинг на логове и контрол върху потребителските сесии.
- ASP.NET Identity осигурява сигурна автентикация и поддържа възможности за управление на роли и достъп.
- Системата трябва да се обновява периодично чрез прилагане на миграции, обновяване на NuGet пакети и следене за критични уязвимости.
- Препоръчва се внедряване на логиране чрез Serilog или подобен инструмент за диагностика и одит.

- Контролът на достъпа до чувствителни ресурси се осъществява чрез middleware и Role-based Authorization.

5.10.4. Изисквания към апаратното осигуряване

- За локална разработка: компютър с минимум 8 GB RAM, 4-ядрен процесор, Windows 10/11, Visual Studio 2022, SQL Server Express.
- За сървърна инсталация: поддържан от Azure App Service или собствен Windows Server с IIS, .NET Hosting Bundle и SQL Server.
- Клиентите могат да използват всякакви устройства с браузър, поддържащ HTML5, CSS3 и JavaScript (Chrome, Firefox, Edge).
- Няма специфични изисквания към мрежата, но се препоръчва стабилна интернет връзка за достъп до облачната инфраструктура.

6. Тестване

За гарантиране на надеждността на системата е проведено модулно тестване върху основни компоненти от бизнес логиката. Unit тестовете са разработени с помощта на xUnit, а за изолиране на зависимости е използвана библиотеката Moq. Всеки метод от сървисния слой е тестван отделно, като при нужда са използвани mock обекти на репозитории и външни зависимости.

Тестовете обхващат ключови функционалности като:

- Кандидатстване по обява – проверка дали заявката се съхранява коректно;
- Запазване на обява в любими – валидация на съществуващ потребител и обява;
- Зареждане на профил – гарантиране, че при заявка се връща правилен профилен модел;
- Филтриране на обяви – проверка на филтриран списък според подадени параметри.

Всеки unit тест съдържа етапи на Arrange, Act и Assert, и проверява както очакваните резултати, така и гранични случаи и грешки (например при null стойности или невалиден идентификатор).

Примерни тестове:

```
using JobBoard.Controllers;

using JobBoard.Dtos;

using JobBoard.Security;

using JobBoard.Services;

using Microsoft.AspNetCore.Identity;

using Xunit;

using Moq;

using Microsoft.AspNetCore.Mvc;

using System.Threading.Tasks;

namespace JobBoard.tests
{
    public class AccountControllerTests
    {
```

```

        private readonly Mock<IAccountService>
        _accountServiceMock;

        private readonly Mock<UserManager<UserData>>
        _userManagerMock;

        private readonly Mock<SignInManager<UserData>>
        _signInManagerMock;

        public AccountControllerTests()
        {
            _accountServiceMock = new Mock<IAccountService>();

            var userStoreMock = new
            Mock<IUserStore<UserData>>();

            _userManagerMock = new
            Mock<UserManager<UserData>> (
                userStoreMock.Object, null, null, null, null,
                null, null, null, null
            );

            var contextAccessorMock = new
            Mock<Microsoft.AspNetCore.Http.IHttpContextAccessor>();

            var userPrincipalFactoryMock = new
            Mock<IUserClaimsPrincipalFactory<UserData>>();

            _signInManagerMock = new
            Mock<SignInManager<UserData>> (
                _userManagerMock.Object,
                contextAccessorMock.Object,
                userPrincipalFactoryMock.Object,
                null, null, null, null
            );
        }

```

```

    }

    [Fact]

    public async Task
    LogIn_Post_SuccessfulLogin_RedirectsBasedOnRole()

    {

        // Arrange

        var loginModel = new LogInRequest

        {

            Username = "testuser",

            Password = "password"

        };

        _signInManagerMock

            .Setup(s =>
s.PasswordSignInAsync(loginModel.Username,
loginModel.Password, false, false))

            .ReturnsAsync(SignInResult.Success);

        var user = new UserData

        {

            UserName = "testuser",

            AccountTypeId = 2 // Assume role 2 = applicant

        };

        _userManagerMock

            .Setup(u =>
u.FindByNameAsync(loginModel.Username))

```

```

        .ReturnsAsync(user);

var controller = new AccountController(
    _accountServiceMock.Object,
    _userManagerMock.Object,
    _signInManagerMock.Object
);

// Act

var result = await controller.LogIn(loginModel);

// Assert

var redirectResult =
Assert.IsType<RedirectToActionResult>(result);

Assert.Equal("Applicant",
redirectResult.ActionName);

Assert.Equal("CandidateDashboard",
redirectResult.ControllerName);
}

[Fact]

public async Task
Logout_CallsSignOutAndRedirectsToHomeIndex()
{
    // Arrange

    _signInManagerMock
        .Setup(s => s.SignOutAsync())
        .Returns(Task.CompletedTask);

```

```

var controller = new AccountController(
    _accountServiceMock.Object,
    _userManagerMock.Object,
    _signInManagerMock.Object
);

// Act

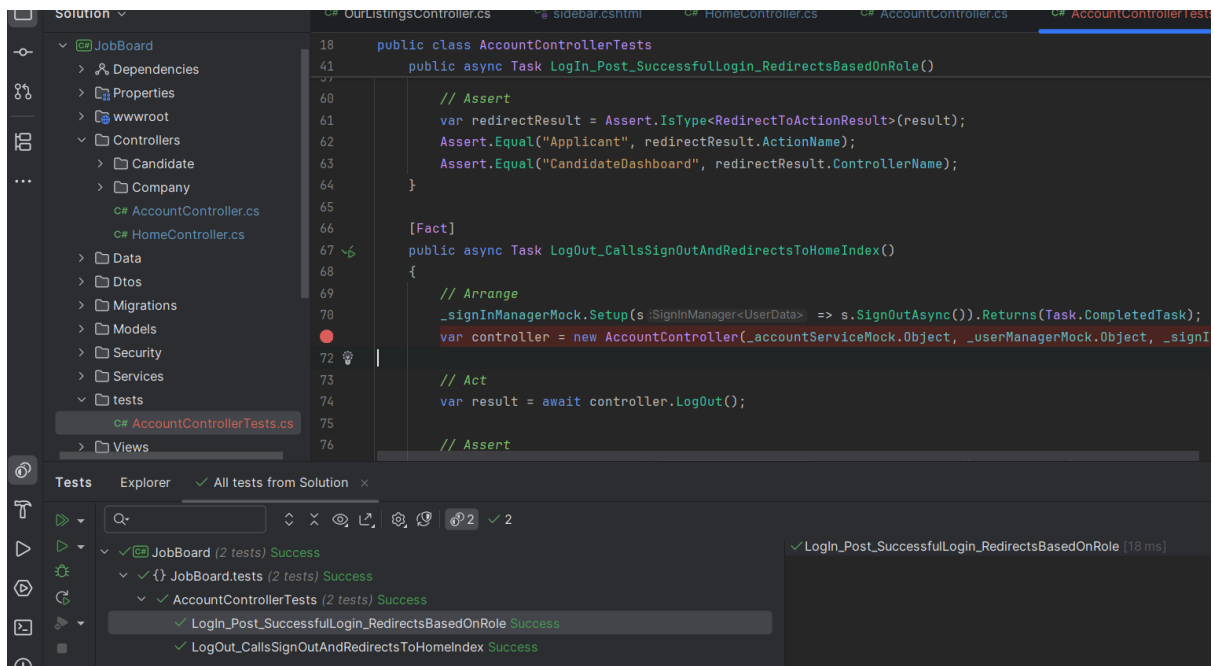
var result = await controller.LogOut();

// Assert

_signInManagerMock.Verify(s => s.SignOutAsync(), Times.Once);

var redirectResult = Assert.IsType<RedirectToActionResult>(result);
Assert.Equal("Index", redirectResult.ActionName);
Assert.Equal("Home", redirectResult.ControllerName);
}
}
}

```



7. Използвана литература

- [1]. <https://www.jobs.bg> – Преглед на структура и съдържание на реални обяви за работа.
- [2]. <https://www.dev.bg> – Идеи за функционалности, категории и UX оформление.
- [3]. <https://learn.microsoft.com/en-us/aspnet/core/> – Официална документация за ASP.NET Core MVC.
- [4]. <https://learn.microsoft.com/en-us/ef/core/> – Официална документация за Entity Framework Core.
- [5]. <https://www.jetbrains.com/rider/> – JetBrains Rider IDE – използвана като среда за разработка на .NET приложения.
- [6]. <https://www.w3schools.com/> – Референции за HTML5, CSS3 и JavaScript при изграждане на потребителския интерфейс.
- [7]. <https://docs.microsoft.com/en-us/azure/> – Инструкции и документация относно хостване в Azure.